

WIMS-BFP API Reference

Compliance-Aware Developer Reference

2026-07-19

Contents

WIMS-BFP API Reference	1
1. Overview	2
2. Authentication & Authorization	2
2.1 Login flow	2
2.2 Token transport and validation — verified against code	3
2.3 Role model	4
2.4 Session revocation	5
2.5 The public / unauthenticated surface	6
3. Error Model	6
3.1 Rate-limiting posture varies by endpoint — fail-open vs. fail-closed	7
4. Endpoint Reference	8
4.1 Auth & Session	8
4.2 Civilian / Public Intake	11
4.3 Regional Encoder	16
4.4 Validator / Triage / Perimeters	18
4.5 Analytics	21
4.6 Incidents / Operations / Dashboard / Events	23
4.7 Admin	25
5. Data Handling & Compliance Appendix	28
5.1 PII map — which endpoints touch personal data	28
5.2 Verified security behavior — PII decryption fails closed	30
5.3 Data classification — encrypted vs. plaintext boundary	31
5.4 Audit logging coverage	31
5.5 Auth mechanism summary (cross-reference to §2)	32
6. Follow-Up Work (deferred, out of scope for this PR)	32
7. Existing Spec Status	33
8. Endpoints Intentionally Excluded	33

WIMS-BFP API Reference

Compliance-aware developer reference. Generated from the live source tree at commit `e792e15590b48b905cbd2433fc48ca8904cbdfcb` (branch `master`). This document describes the system **as implemented**, not as designed or aspired to — every claim below was verified by reading the actual route handlers, Pydantic schemas, and dependency functions in `src/backend/`, not by reading comments, docstrings, or `CLAUDE.md` in isolation.

Scope note. This reference deliberately omits a subset of internal/operational endpoints (backup/restore tooling, LLM/Ollama-driven analysis endpoints, raw infrastructure metrics, and two deprecated 410 Gone stubs). See §8, “Endpoints Intentionally Excluded,” for the full list and rationale. Publishing full request/response contracts for that surface would document internal attack surface with no benefit to external consumers or compliance reviewers.

1. Overview

WIMS-BFP is a Dockerized incident-management system for the Philippine Bureau of Fire Protection. The backend is a FastAPI application (`src/backend/main.py`) with 211 route handlers across 42 files under `src/backend/api/routes/`.

API versioning — current state, documented honestly: the FastAPI app is constructed as:

```
app = FastAPI(title="WIMS-BFP Backend")
```

No `version=`, `description=`, or custom `docs_url=/redoc_url=/openapi_url=` are set. This means: - `/docs` (Swagger UI), `/redoc`, and `/openapi.json` are live at their FastAPI defaults. - There is **no API version string** exposed anywhere (no `/v1` global prefix, no version field in responses). One route group, `public_dmz.py`, does use an explicit `/api/v1/public` prefix — but this is not a repo-wide convention; every other domain is mounted directly under `/api/...` with no version segment. - Because `response_model` is inconsistently applied (see §7), the live `/openapi.json` is a **usable structural scaffold but not a complete contract** — request bodies are generally well-typed (Pydantic body parameters populate the schema even without `response_model`), but roughly half the routes (validator, analytics, most of admin) return bare dicts with no schema, so their response shape in `/openapi.json` renders as `{}`. This document’s endpoint reference fills that gap by hand-documenting response shapes read directly from each handler’s return statement.

Base URL: not fixed in code; determined by deployment (`NEXT_PUBLIC_API_URL` on the frontend side). All paths in this document are relative, e.g. `POST /api/civilian/reports`.

Intake architecture (for context — see `CLAUDE.md` for the full picture): there are two parallel unauthenticated-to-authenticated pipelines feeding official incidents — the zero-trust **Public DMZ** (`/api/v1/public/report`, direct write, no staging) and the **Civilian staging** pipeline (`/api/civilian/reports` → triage cluster queue → validator promotion → `wims.fire_incidents`). Regional encoders separately import AFOR workbooks. See §4 for the full per-domain breakdown.

2. Authentication & Authorization

2.1 Login flow

WIMS-BFP uses Keycloak via OAuth2 **PKCE**. The frontend performs the authorization-code exchange itself but hands the code + `code_verifier` to the backend, which completes the token exchange server-side:

POST /api/auth/callback (public – this IS the login endpoint)

Request: AuthCallbackRequest { code: str, code_verifier: str, redirect_uri: str | None }. The handler exchanges these with Keycloak’s token endpoint, validates the resulting JWT (see §2.2), upserts a wims.users row (JIT-provisioning for non-privileged roles only – see below), and returns { access_token, refresh_token, user_id }.

JIT provisioning guard: JIT_PRIVILEGED_ROLES = frozenset({"SYSTEM_ADMIN"}) blocks auto-provisioning of admin accounts through this path – a token carrying SYSTEM_ADMIN but no matching wims.users row is rejected with 403 and an audit row (JIT_PROVISION_BLOCKED). SYSTEM_ADMIN accounts must be created explicitly via POST /api/admin/users.

Rate limit: a dedicated Redis sliding-window limiter (main.py rate_limit_middleware) caps this endpoint at **5 requests / 15 minutes per client IP** (resolved via X-Real-IP, never the client-controlled X-Forwarded-For). Fails **closed:** a Redis outage returns 503 with Retry-After: 30, unless RATE_LIMIT_FAIL_OPEN=true (dev-only escape hatch).

2.2 Token transport and validation – verified against code

The access token is carried in an **HttpOnly cookie**, never an Authorization header. The cookie is read literally as:

```
token = request.cookies.get("access_token")
```

(src/backend/auth.py:326 in optional_auth, :346 in get_current_user). CSRF is mitigated separately via the __Host- cookie-name prefix, SameSite=Strict, and Origin/Referer validation middleware.

Verified finding – JWKS/issuer split. This is deliberately documented here because it will be stated as fact in a compliance-facing doc, and the request explicitly required verification against code rather than comments. Quoting src/backend/auth.py:26-42:

```
KEYCLOAK_REALM_URL = os.environ.get(
    "KEYCLOAK_REALM_URL",
    os.environ.get("KEYCLOAK_URL", "http://localhost:8080/auth/realms/bfp"),
)
...
# Issuer URL as it appears in JWT `iss` claim – differs from KEYCLOAK_REALM_URL
# when Keycloak is accessed externally (browser → localhost:8080) vs internally
# (container network → keycloak:8080). Set KEYCLOAK_ISSUER so jwt.decode()
# validates the token's iss claim against the browser-visible issuer.
KEYCLOAK_ISSUER = os.environ.get("KEYCLOAK_ISSUER",
KEYCLOAK_REALM_URL.rstrip("/") + "/")
```

The JWKS (signing keys) are fetched from KEYCLOAK_REALM_URL – the **Docker-internal** host (keycloak:8080 inside the compose network) – via OIDC discovery. If discovery itself returns a localhost-based jwks_uri (Keycloak’s own default self-description), the backend rewrites it back to the internal host before fetching (auth.py:125-132, “Docker networking fix”):

```
if "localhost" in jwks_uri:
    jwks_uri = jwks_uri.replace(
```

```

        "http://localhost:8080",
        KEYCLOAK_REALM_URL.rsplit("/auth/realms", 1)[0],
    )

```

The JWT's iss claim is then validated against KEYCLOAK_ISSUER — the **browser-visible** host (localhost:8080 in dev, the public domain in prod) — via an explicit issuer= argument to jwt.decode() (auth.py:209-219):

```

payload = jwt.decode(
    token,
    pem_key,
    algorithms=["RS256"],
    audience=AUDIENCE,
    issuer=KEYCLOAK_ISSUER,
    options={
        "verify_at_hash": False,
        "require": ["exp", "iat", "iss", "aud"],
    },
)

```

What this means in practice: the backend fetches its trust anchor (public signing keys) from one hostname and validates the token's self-declared issuer string against a *different* hostname. This is not a security weakness in itself — the two env vars (KEYCLOAK_REALM_URL, KEYCLOAK_ISSUER) are independently operator-configured, and a token forged with a mismatched iss is still rejected by signature verification against the fetched JWKS. It is a **deliberate accommodation for Docker's internal/external hostname split**, not a shortcut or an oversight — the code comment states this intent directly, and the behavior matches. Anyone deploying WIMS-BFP behind a different network topology should audit both env vars together, since misconfiguring either independently is possible without immediate errors (a wrong KEYCLOAK_ISSUER simply rejects all tokens; the failure mode is fail-closed, not silent).

After signature + issuer + audience validation, the handler also checks azp (authorized party) matches the expected CLIENT_ID, and performs an **instant revocation check** (§2.4) before trusting the payload.

2.3 Role model

Roles are resolved from the Keycloak token's realm_access.roles (or client-scoped resource_access.<client>.roles) using a fixed precedence tuple, WIMS_ROLES_FROM_KEYCLOAK (auth.py:45-51):

```

WIMS_ROLES_FROM_KEYCLOAK = (
    "CIVILIAN_REPORTER",
    "REGIONAL_ENCODER",
    "NATIONAL_VALIDATOR",
    "NATIONAL_ANALYST",
    "SYSTEM_ADMIN",
)

```

Listed ascending by privilege; resolution walks the tuple and keeps the highest-privilege match, so a user token carrying multiple roles (e.g. via a default-roles composite) resolves to its single highest role rather than the first match.

Five FastAPI dependency functions gate routes by role (all built on top of `get_current_wims_user`, which itself resolves the JWT to a `wims.users` row and requires `is_active = TRUE`):

Dependency	Condition	Roles admitted
<code>get_current_wims_user</code>	valid JWT + active wims.users row	any authenticated user
<code>get_regional_encoder</code>	role in ("REGIONAL_ENCODER", "ENCODER")	Regional Encoder
<code>get_national_validator</code>	role == "NATIONAL_VALIDATOR"	National Validator
<code>get_national_analyst</code>	role == "NATIONAL_ANALYST"	National Analyst (strict)
<code>get_analyst_or_admin</code>	role in ("NATIONAL_ANALYST", "SYSTEM_ADMIN")	Analyst dashboards, broad- ened for admin
<code>get_system_admin</code>	role == "SYSTEM_ADMIN"	System Admin

A handful of route files use **local, file-specific role predicates** layered on `get_current_wims_user` rather than the table above — these are called out explicitly in each domain section below because they are deviations worth knowing about, not because they're wrong: - `regional/validator.py`'s correction endpoint accepts `NATIONAL_VALIDATOR` **or** `NATIONAL_ANALYST`. - `regional/perimeters.py` defines its own `_require_perimeter_editor` (`NATIONAL_VALIDATOR/SYSTEM_ADMIN`) and `_require_perimeter_reader` (adds `REGIONAL_ENCODER` for read-only access). - `trriage.py` defines three local predicates (`_require_encoder_or_validator`, `_require_cluster_workflow_actor`, `_require_status_update_actor`) for its cluster-workflow and civilian-status-update surface. - `map.py`'s `operational_router` (mounted at `/api/validator`) declares **no explicit role dependency** in its route signatures — only an RLS-scoped DB session (`get_db_with_rls`). This is flagged in §4.4 as worth verifying against any router-level dependencies= set at mount time in `main.py`, since `CLAUDE.md`'s documented convention is `get_current_wims_user` before `get_db_with_rls` on every gated route. - `admin/sync.py`'s `POST /sync/report` intentionally uses only `get_current_wims_user` (any authenticated role) — it's a one-way client-side sync-failure beacon, not an admin action, despite living under `/api/admin`.

2.4 Session revocation

Deactivating a user (`PATCH /api/admin/users/{id}` with `is_active=false`) triggers instant revocation via Redis, independent of JWT expiry (`src/backend/utils/session.py`): - Key

pattern: `revoked_user:{keycloak_id}` → value is the Unix timestamp of revocation. - TTL: 43200 seconds (12 hours) — chosen to outlast any issued token’s natural lifetime. - On every authenticated request, `is_token_revoked(keycloak_id, iat)` compares the token’s iat against the stored revocation timestamp; a token issued before revocation is rejected instantly rather than waiting out its exp. - **Fails open** if Redis is unreachable (returns “not revoked” rather than blocking all traffic) — a deliberate availability trade-off distinct from the fail-closed posture used elsewhere (public DMZ rate limiting, PII encryption).

2.5 The public / unauthenticated surface

The following endpoints carry **no** JWT requirement by design (full detail in §4):

POST `/api/v1/public/report`, POST `/api/civilian/reports` (+ its `append/photos/followup/notify/duplicate-suggestions/tracking` sub-routes — anonymous-capability-gated, not JWT-gated), POST `/api/auth/register`, POST `/api/auth/verify-registration`, POST `/api/auth/security-event`, POST `/api/auth/consent`, POST `/api/auth/callback`, GET `/api/community/hub + /{slug}`, GET `/api/information/announcements + /emergencies`, GET `/api/public/clusters + /emergency-services`, GET `/api/ref/fire-stations + /emergency-services`, GET `/api/geocode/reverse + /search`, GET `/api/civilian/reports/{id}/track/{token}`, GET `/api/civilian/report-clusters`.

POST `/api/auth/keycloak-event` is also unauthenticated by JWT but is **not** a client-facing endpoint — it’s a server-to-server ingest secured by a shared Bearer secret (`WIMS_KEYCLOAK_EVENT_SECRET`), called only by the Keycloak SPI event listener. See §8.

3. Error Model

WIMS-BFP does not use a single centralized error envelope; conventions are consistent enough across domains to document once:

- **HTTPException(status_code, detail=...)** is FastAPI’s native mechanism and is used throughout; detail strings pass through verbatim to the client — the global exception handler (`main.py`) only intercepts *unhandled* exceptions, converting them to a generic 500 `{"detail": "An unexpected error occurred..."}`. It does not override FastAPI’s built-in HTTPException handling, so any 4xx raised deliberately by a route keeps its specific message.
- **401** — missing/invalid/expired credentials (bad signature, wrong issuer/audience, revoked session).
- **403** — authenticated but wrong role, or an object-level ownership check failed (region mismatch, wrong encoder).
- **404** — **used deliberately as a “neutral” response** in several unauthenticated flows, to avoid confirming whether a resource exists. Two concrete, code-quoted examples:
 - Civilian report device-ownership check (`civilian.py::_require_device_ownership`):
“Civilian report routes are intentionally unauthenticated, so the `device_id` token is the object-level authorization boundary. Missing, unknown, and wrong-device accesses all return the same neutral 404 shape.”

- Tracking-token validation (`civilian.py`, both the claim and public-tracking routes): # Neutral 404 – do not reveal whether the report exists or the token is wrong. → `HTTPException(404, "Report not found")` regardless of which check failed.
- **409** – conflict/idempotency states: optimistic-concurrency-control (OCC) version mismatches on incident edits, duplicate-detection holds on submission, already-linked/already-applied replay states.
- **422** – Pydantic validation failures, plus several handlers’ own semantic validation (invalid enum-like string params, malformed UUIDs, bundle-size caps).
- **429** – rate limits, always paired with a bounded `Retry-After` header where Redis-backed (see §3.1 below for the fail-open/fail-closed distinction, which varies meaningfully by endpoint).
- **503** – an upstream dependency (Redis, Keycloak, OpenBao/KMS) is unreachable, on the endpoints where the design choice is to fail closed rather than degrade silently.
- **410** – permanently retired endpoints. Two confirmed in this codebase snapshot:
 - `POST /api/triage/{report_id}/promote` – unconditionally returns 410 Gone.
 - `GET /api/civilian/contributor/leaderboard` – unconditionally returns 410 Gone (“The contributor leaderboard has been removed”).
 - `GET /api/civilian/reports` (the bare list, no path params) – unconditionally returns 410 Gone (“This legacy tracking endpoint has been retired. Use the secure tracking link.”).
 - **Caveat surfaced by the research pass, not smoothed over:** `POST /api/triage/bulk-promote` was originally assumed to be a matching 410 stub alongside the single-report promote endpoint. On inspection, **it is not** – `bulk_promote_reports` is a live, functional handler (`_bulk_set_status(..., action_type="CIVILIAN_REPORT_PROMOTE")`) that sets report status to ACTIONED. Only `POST /{report_id}/promote` contains the literal `raise HTTPException(410, ...)`. This document lists `/bulk-promote` under its actual (functioning) behavior in §4.5 and flags this discrepancy for the team to reconcile – either the bulk path should also be retired for consistency, or the singular path’s deprecation notice should be corrected.

3.1 Rate-limiting posture varies by endpoint – fail-open vs. fail-closed

Not all rate limiters degrade the same way on a Redis outage. This distinction is compliance-relevant (it determines whether an outage widens or narrows abuse exposure) and was verified per-endpoint rather than assumed uniform:

Endpoint / layer		Backing		On Redis failure
POST	/api/v1/public/report (public DMZ, primary limiter)	Redis	sliding-window script	Lua Connect failure → fail-open (no limiting applied); Redis reachable but the eval call errors → fail-closed , 503
POST	/api/civilian/reports, /append, /reports/{id}/followup	Postgres	COUNT(*) pg_advisory_xact_lock (not Redis)	+ N/A — DB-backed, no external fail-open/closed distinction; the advisory lock exists specifically to close a TOC-TOU race (issue #446 gap #14 / P0-2)
POST	/api/civilian/reports/{id}/notify	Redis	sliding-window (rate_limit_public(..., fail_closed=True))	Fail-closed , 503
Device-abuse Tier 2 (anonymous civilian submissions)		Redis	sliding-window (rate_limit_public(..., fail_closed=False))	Fail-open — deliberately looser, since this is a secondary abuse layer behind the primary DB-backed cap
POST	/api/auth/callback (login)	Redis	sliding-window, dedicated middleware	Fail-closed , 503 + Retry-After: 30 (unless RATE_LIMIT_FAIL_OPEN=true, dev-only)
POST	/api/auth/consent	Redis,	rate_limit_public(..., limit=5, window=3600)	Fail-closed per design note “D6”

4. Endpoint Reference

Grouped into the domains used throughout this document: Auth & Session, Civilian / Public Intake, Regional Encoder, Validator / Triage / Perimeters, Analytics, Incidents / Operations / Dashboard / Events, and Admin.

Every entry states its auth requirement explicitly as **PUBLIC** or the exact dependency/role gating it. Response shapes for endpoints lacking `response_model` were hand-documented by reading the handler’s return statement — not inferred, not assumed.

4.1 Auth & Session

POST /api/auth/callback

- **Auth:** PUBLIC (this is the login endpoint)
- **Purpose:** Completes the OAuth2 PKCE handshake — exchanges code+code_verifier with Keycloak, validates the JWT, upserts `wims.users` (JIT-provisions non-privileged roles only).

- **Request:** AuthCallbackRequest { code: str, code_verifier: str, redirect_uri: str | None }
- **Response:** { access_token: str, refresh_token: str | None, user_id: str }
- **Notable errors/rate limits:** 401 token exchange failed / missing access_token / missing sub claim; 403 no resolvable WIMS role, or JIT privilege guard blocks an unenrolled SYSTEM_ADMIN token (writes JIT_PROVISION_BLOCKED audit); 500 user_id upsert failed. Rate limit: 5 req / 15 min / IP, fail-closed (503, Retry-After: 30).

POST /api/auth/change-email

- **Auth:** Depends(get_current_wims_user)
- **Purpose:** Step 1 of email change — verifies current password (+optional TOTP) against Keycloak, stores a pending change + 6-digit code in Redis (10-min TTL), emails the code.
- **Request:** ChangeEmailRequest { new_email: EmailStr, current_password: str, otp_code: str | None }
- **Response:** { status: "ok", message: str }
- **Notable errors/rate limits:** 503 Redis unavailable; 429 (3 req / 10 min / user); 400 blank password; 401 wrong password/OTP; 502 email send failure (rolls back Redis key).

POST /api/auth/verify-email

- **Auth:** Depends(get_current_wims_user), Depends(get_db_with_rols)
- **Purpose:** Step 2 — verifies the 6-digit code, updates email in Keycloak + wims.users.
- **Request:** VerifyEmailRequest { code: str }
- **Response:** { status: "ok"|"partial", message: str } ("partial" = Keycloak succeeded, DB sync failed)
- **Notable errors/rate limits:** 503 Redis down; 429 (5 attempts / 10 min); 400 blank/mismatched code; 404 no pending change; 500 corrupt Redis state; 502 Keycloak update failed.

POST /api/auth/register

- **Auth:** PUBLIC — Turnstile CAPTCHA + IP rate limit instead of JWT
- **Purpose:** Civilian self-service signup (verify-first). Creates a **disabled** Keycloak user with CIVILIAN_REPORTER role; no DB record yet.
- **Request:** CivilianRegisterRequest { email: EmailStr, first_name: str, last_name: str, password: str, contact_number: str (11 chars), dpa_consent: bool, turnstile_token: str }
- **Response:** RegisterResponse { status, message, email, user_id: str | None } (201)
- **Notable errors/rate limits:** 429 IP limit or Turnstile rejection; 503 Keycloak/Redis unavailable (rolls back the disabled Keycloak user); 409 email already exists; 400/502 Keycloak creation failure; 502 verification email failure. Rate limit: 3 req/hour/IP.

POST /api/auth/verify-registration

- **Auth:** PUBLIC
- **Purpose:** Finalizes registration — validates code, enables the Keycloak account, creates wims.users + wims.civilian_contributors. Logs DPA_CONSENT audit event if consent was given.

- **Request:** VerifyRegistrationRequest { email: str, code: str }
- **Response:** VerifyRegistrationResponse { status, message }
- **Notable errors/rate limits:** 503 Redis/Keycloak down; 400 bad code; 404 no pending registration or drifted Keycloak account; 500 corrupt Redis state; 502 Keycloak/DB failure (rolls back with best-effort Keycloak cleanup). 5 attempts / 10 min / email.

POST /api/auth/security-event

- **Auth:** PUBLIC — Keycloak owns the credential check; this only records non-repudiation audit
- **Purpose:** Records a client-reported auth-lifecycle event (FAILED_LOGIN/PASSWORD_RESET/LOGOUT) into system_audit_trails, since these happen inside Keycloak and never otherwise reach the backend.
- **Request:** SecurityEventRequest { event_type: str, username: str | None }
- **Response:** { status: "recorded", event_type } (202)
- **Notable errors/rate limits:** 422 unsupported event_type. 30 req/60s/IP, best-effort (skipped silently if Redis down).

POST /api/auth/consent

- **Auth:** PUBLIC
- **Purpose:** Records a data-subject consent grant/withdrawal into wims.consent_log + audit trail (CONSENT_GRANT/CONSENT_WITHDRAW).
- **Request:** ConsentRequest { subject_type: "USER"|"REPORT", subject_id: str, consent_type: str, action: "GRANTED"|"WITHDRAWN" }
- **Response:** ConsentRecord { consent_id, subject_type, subject_id, consent_type, action, recorded_at } (201)
- **Notable errors/rate limits:** 5 req/hour/IP, fail-closed. Client IP is stored only as a salted SHA-256 hash, never raw.

POST /api/auth/keycloak-event — see §8 (excluded, internal ingest)

POST /api/auth/consent, GET /api/community/hub etc. — cross-referenced in their own domain sections; not duplicated here.

GET /api/admin/sessions/{user_id}

- **Auth:** Depends(get_system_admin)
- **Purpose:** Lists active Keycloak sessions for a user (addressed by internal WIMS user_id).
- **Response:** { sessions: [...] }. **Errors:** 404 unresolvable user.

DELETE /api/admin/sessions/{user_id}

- **Auth:** Depends(get_system_admin)
- **Purpose:** Terminates **all** active Keycloak sessions for a user; writes a LOGOUT audit row.
- **Response:** { status: "ok", user_id }. **Errors:** 404.

DELETE /api/admin/sessions/{user_id}/{session_id}

- **Auth:** Depends(get_system_admin)
- **Purpose:** Revokes one specific session; writes a LOGOUT audit row.

- **Response:** { status: "ok", session_id }. **Errors:** 404 session not found; 500 deletion failure.

GET /api/user/me/profile

- **Auth:** Depends(get_current_wims_user)
- **Purpose:** Caller's full profile — Keycloak identity fields merged with contact_number/email_opt_in/push_opt_in from wims.users.
- **Response:** profile dict (no fixed schema; merges Keycloak + DB fields).

PATCH /api/user/me

- **Auth:** Depends(get_current_wims_user)
- **Purpose:** Self-service profile update (first_name, last_name, contact_number, notification prefs). Role/region are not editable here; email changes must go through the change-email flow.
- **Request:** ProfileUpdate (all optional) { first_name, last_name, email, current_password, contact_number (^09\d{9}\$), email_opt_in, push_opt_in }
- **Response:** { status: "ok"|"partial", message }
- **Notable errors:** 400 no fields, or attempted direct email change; 502 both DB and Keycloak sync failed.

PATCH /api/user/me/password

- **Auth:** Depends(get_current_wims_user)
- **Purpose:** Self-service password change; verifies current password via a distinct Direct-Grant Keycloak client (bfp-client, not the browser's wims-web), then logs out all sessions.
- **Request:** PasswordChange { current_password, new_password (≥8 chars, upper+digit+special), otp_code: str | None }
- **Response:** { status: "ok", message }. **Errors:** 401 wrong password/OTP; 502 Keycloak update failure.

Excluded from full documentation: GET /health, GET /metrics (both include_in_schema=False), POST /api/auth/keycloak-event — see §8.

4.2 Civilian / Public Intake

The flagship unauthenticated surface. Covers public_dmz.py, civilian.py, the public halves of community_content.py/information.py/map.py, ref.py, and geocode.py.

POST /api/v1/public/report — zero-trust direct submission

- **Auth:** PUBLIC — no JWT, no RLS context. encoder_id is always NULL.
- **Purpose:** Submit a fire incident directly into wims.fire_incidents, no staging step. Region resolved from coordinates via nearest fire station.
- **Request:** PublicIncidentCreate { latitude (-90..90), longitude (-180..180), description (1..2000 chars) }
- **Response:** PublicIncidentResponse { incident_id, latitude, longitude, verification_status, created_at } (201)

- **Rate limit** — **exactly 3 requests per IP per hour**, Redis sliding-window sorted-set Lua script, keyed `public_rate_limit:{ip}`. IP source is X-Real-IP only (nginx-set, never client-controlled X-Forwarded-For).
 - **Fail behavior is split, not uniformly closed or open:** Redis unreachable *at connect time* → fail-open (no limiting for that request); Redis reachable but the Lua eval call itself errors → **fail-closed**, 503 "Service temporarily unavailable – rate limiter unreachable", plus a `wims_public_ratelimit_failclosed_total` Prometheus counter increment.
 - 429 on limit exceeded, with a Retry-After header bounded 1–3600s.
- **Notable errors:** 500 no region reference data resolvable; 500 insert failed / audit-trail insert failed (the `fire_incidents` INSERT and the audit INSERT share one transaction — audit failure rolls back both, per design note D20/#394); 500 post-insert coordinate re-read failed. Client IP is stored only as a salted hash in the audit row.

GET /api/civilian/reports (bare list)

- **Auth:** PUBLIC — **410 Gone** unconditionally: "This legacy tracking endpoint has been retired. Use the secure tracking link."

POST /api/civilian/reports

- **Auth:** `optional_auth` — PUBLIC by default (anonymous, subject to CAPTCHA + device-abuse tiers); a valid `CIVILIAN_REPORTER` JWT upgrades the rate cap and skips CAPTCHA. `optional_auth` is fail-closed on an *invalid* token (raises) — only a fully absent credential is treated as anonymous.
- **Purpose:** Submit a new civilian report into the staging table, resolve nearest station, compute a trust score, encrypt witness PII, issue a tracking token, enqueue async routing.
- **Request:** `CivilianReportCreate` { `latitude`, `longitude`, `category`: `STRUCTURAL|NON_STRUCTURAL|TRANSPORTATION|UNSURE`, `sub_category`, `reported_at`, `device_id`, `reporting_context`: `WITNESS|NEARBY|SECONDHAND`, `safety_status`: `I_AM_SAFE|I_NEED_HELP|SOMEONE_ELSE_NEEDS_HELP|UNKNOWN`, `phone_latitude/longitude`, `gps_distance_m`, `gps_warning_confirmed`, `witness_name`, `witness_phone`, `previous_report_id`, `source_url`, `client_report_id`, `turnstile_token` }
- **Response:** `CivilianReportResponse` { `report_id`, `latitude`, `longitude`, `category`, `sub_category`, ..., `trust_score`, `status`, `guidance`, `nearest_station_name/phone`, `routing_*`, `photo_count`, `tracking_token`, `tracking_url`, `link_count`, `created_at` } (201, or 200 on idempotent replay)
- **Notable errors/rate limits:**
 - 422 malformed `client_report_id`; 404 `previous_report_id` not found.
 - **Rate cap: 3 req/IP/hour anonymous, 20 req/IP/hour for authenticated CIVILIAN_REPORTER** — DB-backed (`COUNT(*)` on `wims.citizen_reports` by `ip_hash`), guarded by `pg_advisory_xact_lock` to close a TOCTOU race (P0-2/#446 gap #14). 429 with bounded Retry-After.
 - Idempotency via `client_report_id`: an early match returns the existing report with 200 (no quota consumed); a DB-level `ON CONFLICT ... DO NOTHING` is the TOCTOU safety net.

- Witness PII encryption failure is **not** fail-closed here — logs a warning and falls through to plaintext columns, so a KMS outage cannot block civilian intake.
- Device quarantine (issue #572): flagged submissions get `requires_review=true` forced — never blocked, only routed for mandatory validator review.
- Tracking token: `secrets.token_hex(32)`, stored only as its SHA-256 hash; the raw token is returned once and never persisted.

POST /api/civilian/reports/claim

- **Auth:** `optional_auth`, manually enforced to require `role == "CIVILIAN_REPORTER"` (effectively authenticated-only despite the dependency name).
- **Purpose:** Attach an anonymously-submitted report to the caller's account using the tracking token.
- **Request:** `CivilianReportClaim { report_id, tracking_token }`
- **Response:** `CivilianReportResponse`
- **Notable errors:** **Neutral 404** (quoted from code: # Neutral 404 — do not reveal whether the report exists or the token is wrong.) for any invalid/revoked/expired token; 409 if already claimed (`race-safe UPDATE ... WHERE contributor_user_id IS NULL`).

GET /api/civilian/reports/{report_id}/track/{tracking_token}

- **Auth:** `PUBLIC` (`get_public_db_with_ribs`)
- **Purpose:** Read-only tracking page — status, station, coarse routing, photo count, timeline. Deliberately excludes location and PII.
- **Response:** `CivilianTrackingResponse { report_id, category, safety_status, status, guidance, nearest_station_*, routing_*, photo_count, status_updates, created_at }`
- **Notable errors:** Neutral 404 for any invalid/expired/mismatched token — same pattern as the claim route.

POST /api/civilian/reports/duplicate-suggestions

- **Auth:** `PUBLIC`
- **Purpose:** Non-blocking nearby-duplicate suggestions (100m/1hr radius); suppressed entirely for active-emergency safety statuses.
- **Request:** `CivilianReportCreate` (reused schema)
- **Response:** `DuplicateSuggestionResponse { suggestions: [{report_id, distance_m, category, ..., created_at}] }`
- **Notable errors:** none; no rate limiter on this route.

PATCH /api/civilian/reports/{report_id}/append

- **Auth:** `optional_auth` (anonymous goes through device-abuse check)
- **Purpose:** Append a child ("LINKED") report to an active parent.
- **Request:** `CivilianReportAppend` (subset of create fields + required description)
- **Response:** `CivilianReportResponse` (201)
- **Notable errors:** device-ownership neutral-404 gate; 409 if parent status is terminal; 429 if the same device appended within 5 minutes.

POST /api/civilian/reports/{report_id}/followup

- **Auth:** PUBLIC (no CAPTCHA/device-abuse check at all — a narrower surface than append/submit)
- **Purpose:** Free-text follow-up on an existing report.
- **Request:** CivilianFollowupCreate { device_id (1..128), followup_text (1..2000) }
- **Response:** CivilianFollowupResponse { followup_id, report_id, followup_text, created_at } (201)
- **Notable errors/rate limits:** device-ownership neutral-404; 409 terminal report; **429 at 5/report/IP/hour and 429 at 10/all-reports/IP/hour** — both DB-backed with the same advisory-lock TOCTOU guard as the main submit route.

POST /api/civilian/photos/upload, POST /api/civilian/reports/{report_id}/photos

- **Auth:** optional_auth (+ get_anonymous_session_id for the pending-upload variant). Registered CIVILIAN_REPORTER: 5 photos / 10 MiB; anonymous via device_id: 1 photo / 5 MiB.
- **Purpose:** Upload and attach an encrypted photo to a report (or a pre-report “pending” photo).
- **Request:** multipart — file, browser/EXIF GPS metadata fields, client_photo_id.
- **Response:** PhotoUploadResponse/PendingPhotoUploadResponse { photo_id, report_id, duplicate, file_size_bytes, mime_type, image_width/height, exif_gps_status, browser_gps_status, gps_consensus, photo_reported_distance_m } (201)
- **Notable errors:** 422 on out-of-range EXIF GPS or malformed UUID/datetime fields. Uses a non-superuser DB session with FORCE ROW LEVEL SECURITY on wims.report_photos.

POST /api/civilian/reports/{report_id}/notify

- **Auth:** PUBLIC
- **Purpose:** Register an FCM push token for status-change alerts.
- **Request:** NotifyRegisterRequest { device_id, fcm_token }
- **Response:** { status: "registered"|"already_registered", report_id } (201)
- **Notable errors/rate limits:** device-ownership neutral-404; 429 at 10 tokens/report; **429 at 5 registrations/IP/hour, fail-closed** (503 on Redis outage) — the one civilian endpoint sharing the DMZ-style fail-closed posture rather than fail-open.

GET /api/civilian/contributor/me, /reports, /stats

- **Auth:** get_current_wims_user + manual role == "CIVILIAN_REPORTER" check (403 otherwise)
- **Purpose:** Trust-score breakdown, paginated report history, and vanity stats for the authenticated contributor.
- **Response:** ContributorProfileResponse / ContributorReportsResponse / ContributorStatsResponse.

GET /api/civilian/contributor/leaderboard

- **Auth:** PUBLIC — **410 Gone** unconditionally (“The contributor leaderboard has been removed”).

GET /api/civilian/contributor/{user_id}

- **Auth:** get_national_validator — validator-facing despite living in the civilian router.

- **Purpose:** Validator’s view of an arbitrary contributor’s profile + reports.
- **Response:** ContributorDetailResponse { profile, reports }. **Errors:** 400 malformed UUID.

GET /api/civilian/report-clusters

- **Auth:** PUBLIC
- **Purpose:** Heat-map areas of recent report pressure (local mode: 50 clusters/10km/min 3 reports; national mode: 25 clusters/1hr window/min 10 reports).
- **Response:** ReportClusterResponse { mode, center, radius_m, areas: [{area_id (hashed), latitude, longitude, radius_m, count_bucket, age_bucket}] } — deliberately coarse/bucketed, no raw report data. degraded=true if both DB and stale cache are unavailable (not an HTTP error).

GET /api/community/hub, GET /api/community/{slug}

- **Auth:** PUBLIC (get_public_db_with_rls)
- **Purpose:** Public safety-hub feed / single content item, PUBLISHED and non-expired only. Frontend is contractually required to render title/body as plain text, never dangerouslySetInnerHTML.
- **Response:** CommunityHubResponse / CommunityContentDetailResponse. **Errors:** 404 for missing/unpublished/expired slugs.

GET /api/information/announcements, GET /api/information/emergencies

- **Auth:** PUBLIC
- **Purpose:** Published announcements / emergencies (the latter sourced from verified, civilian-linked incidents, including GeoJSON perimeter when available).
- **Response:** list[AnnouncementResponse] / list[EmergencyResponse]. Access control is entirely the SQL published = TRUE predicate (plus, for emergencies, verification_status='VERIFIED' and an EXISTS civilian-link check).

GET /api/ref/fire-stations, GET /api/ref/emergency-services

- **Auth:** PUBLIC — explicitly documented “No auth — called from the public civilian portal.”
- **Purpose:** Station list (nearest-5 if coordinates given) / national emergency contacts + stations.
- **Response:** station/contact lists; Redis-cached with stale-if-error fallback.

(GET /api/ref/regions|provinces|cities require get_current_wims_user — authenticated lookups, not part of the public surface, included here only for completeness since they share the file.)

GET /api/public/clusters, GET /api/public/emergency-services

- **Auth:** PUBLIC
- **Purpose:** Server-side PostGIS clustering of anonymous civilian pressure reports for the public map — explicitly **not** BFP-confirmed incidents. Module docstring: *“Queries wims.citizen_reports (public signal records), excludes rejected/triaged reports, and returns area-level aggregates — never individual reports or personally identifiable information.”*

- **Response:** ClusterResponse / EmergencyServicesResponse. Redis cache (2min/5min TTL) with best-effort single-flight lock; cache failures degrade to an unguarded live query rather than erroring.

GET /api/geocode/reverse, GET /api/geocode/search

- **Auth:** PUBLIC
- **Purpose:** Server-side proxy to Nominatim (OpenStreetMap) so client coordinates never hit a third party directly.
- **Response:** proxied Nominatim JSON, passthrough.
- **Notable errors:** 503 circuit-breaker-open or concurrency-limit (10 concurrent); 502 oversized response (>5MB) or generic upstream error. No inbound per-IP limit — protection is entirely on the outbound call (retry, circuit breaker).

Cross-cutting pattern — device-abuse three-tier escalation (anonymous civilian submissions only): Tier 1 CAPTCHA (403 on missing/invalid Turnstile token) → Tier 2 adaptive Redis rate limit (5 req/min unblocked, 2 req/min if already flagged; **fail-open** on Redis failure, deliberately looser than the primary DB-backed cap) → Tier 3 quarantine (3 Tier-2 violations in 60 min sets a 24h flag that **never blocks**, only forces requires_review=true + a PUBLIC_QUARANTINED_SUBMISSION audit entry).

4.3 Regional Encoder

All mounted under /api/regional (src/backend/api/routes/regional/__init__.py: APIRouter(prefix="/api/regional", tags=["regional"]), sub-routers contribute no prefix of their own). Default gate is Depends(get_regional_encoder); deviations are called out per endpoint.

GET /api/regional/incidents

- **Auth:** get_regional_encoder
- **Purpose:** List the encoder's own incidents (filterable by category/status/date/archived).
- **Response:** { items, total, limit, offset } — each item decrypts the PII blob per row to populate owner_name/caller_name/caller_number.
- **Touches PII:** yes (decrypts blob). **Audit-logged:** no (read-only).

GET /api/regional/incidents/drafts

- **Auth:** get_regional_encoder — **Touches PII:** no. **Audit-logged:** no.

GET /api/regional/incidents/{incident_id}

- **Auth:** get_current_wims_user (**deviation** — authorization is done in-handler: validators/analysts/admins see any incident, everyone else is restricted to encoder_id = self)
- **Purpose:** Full incident detail (nonsensitive + decrypted sensitive fields, wildland flag, hash-chain integrity, rejection reason).
- **Notable errors:** 404 “Incident not found or access denied” for hidden rows.

- **Touches PII:** yes. **Audit-logged:** conditionally — a hidden-row 404 for a non-validator caller triggers `_check_idor_probe()`, which writes `IDOR_PROBE` to `system_audit_trails` (cross-region enumeration signal).

GET /api/regional/audit-log, POST /api/regional/login-event

- **Auth:** `get_regional_encoder` — the encoder's own activity feed / a `USER_LOGIN` audit beacon.
- **Touches PII:** no. `POST /login-event` is audit-logged (`USER_LOGIN`); the GET is read-only.

POST /api/regional/incidents (create, 201)

- **Auth:** `get_regional_encoder`
- **Request:** `IncidentCreateRequest` — the richest schema in this domain (30+ fields spanning nonsensitive structural data plus PII fields `caller_name`, `caller_number`, `owner_name`, `occupant_name`, `narrative_report`; idempotent via `client_id` UUID).
- **Response:** { `status`: "created", `incident_id`, `verification_status`: "DRAFT", `incident_type_code`, `parent_incident_id` }
- **Notable errors:** 400 missing region; 403 `REGION_MISMATCH`; 422 malformed `client_id`.
- **Touches PII:** yes (encrypts into `pii_blob_enc`). **Audit-logged:** yes — both `system_audit_trails` (`CREATE_INCIDENT`) and `incident_verification_history` (`CREATED_DRAFT`).

PUT /api/regional/incidents/{incident_id}, PATCH /api/regional/incidents/draft/{incident_id}

- **Auth:** `get_regional_encoder`
- **Request:** `IncidentUpdateRequest` (same PII surface, all fields optional, plus `client_updated_at/force_update` for OCC)
- **Notable errors:** 404 not found/not owned; 403 wrong status; 409 OCC conflict (server_version payload includes decrypted PII — this is the one place a 409 response body itself carries PII); 500 on unexpected failure.
- **Touches PII:** yes (decrypt-merge-re-encrypt via the shared `_apply_incident_field_updates` helper). **Audit-logged:** yes (`EDITED` in `incident_verification_history`).

POST /api/regional/incidents/{incident_id}/force-replace, PATCH .../unpend, DELETE /api/regional/incidents/{incident_id}, PATCH .../archive, PATCH .../unarchive, PATCH .../submit

- **Auth:** `get_regional_encoder` throughout.
- Lifecycle transitions (replace-a-pending-row, withdraw-to-draft, soft-delete, archive/unarchive, submit-for-review with duplicate-detection gate — 409 `DUPLICATE_DETECTED` unless `ack_duplicate/force`).
- **Audit-logged:** yes for all except `PATCH .../archive`, which performs a raw `UPDATE` with no `log_system_audit/IVH` call — flagged here as a gap worth closing, not smoothed over. `submit` uniquely writes to *both* `incident_verification_history` and `system_audit_trails` directly.

POST /api/regional/afor/import

- **Auth:** `get_regional_encoder`

- **Purpose:** Upload/parse an AFOR .xlsx/.csv into preview rows (10MB cap, magic-byte + decompression-bomb checks).
- **Response:** `AforParseResponse { total_rows, valid_rows, invalid_rows, rows, form_kind, requires_location }`
- **Notable errors:** 400 for every validation failure category (empty file, magic-byte mismatch, decompression bomb, unparseable file, no data rows, region mismatch); wildland-specific AFOR import is explicitly deprecated (400).
- **Touches PII:** no DB write at this stage (preview only). **Audit-logged:** yes (AFOR_IMPORT_PARSE).

POST /api/regional/afor/commit

- **Auth:** `get_regional_encoder`
- **Request:** `AforCommitRequest { rows, form_kind, resolutions (skip/merge/force per duplicate) }`
- **Response:** `AforCommitResponse { status, batch_id, incident_ids, total_committed }`, or a `{ "status": "DUPLICATE_CHECK_REQUIRED", ... }` short-circuit.
- **Touches PII:** yes (batch writes into `incident_sensitive_details`, encrypted). **Audit-logged:** yes, both AFOR_IMPORT_COMMIT (audit trail) and per-incident `incident_verification_history` rows.

GET /api/regional/incidents/check-duplicate

- **Auth:** `get_regional_encoder` — **Touches PII:** partial (`street_address` only, not one of the four named PII fields). **Audit-logged:** no.

GET /api/regional/validator/stats

- **Auth:** `get_national_validator` (**deviation** — mounted under `/api/regional` but gated for validators) — aggregate counts only, no PII.

GET /api/regional/stats

- **Auth:** `get_regional_encoder` — region + personal summary stats, no PII.

4.4 Validator / Triage / Perimeters

Regional Validator (regional/validator.py)

GET /validator/incidents

- **Auth:** `get_national_validator` — the validator's cross-region review queue. **Touches PII:** no. **Audit-logged:** no.

PATCH /incidents/{incident_id}/verification

- **Auth:** `get_national_validator`
- **Request:** `VerificationActionRequest { action: accept|accept_replace|pending|reject, notes, original_incident_id, client_id }`
- **Notable errors:** 400 unknown action; 403 no `encoder_id` (public DMZ row — validator flow doesn't apply); 404; 409 already at target status.

- **Audit-logged:** yes — one `incident_verification_history` row in the same transaction as the status update (atomic).

PATCH /incidents/{incident_id}/correct

- **Auth:** `get_current_wims_user` + manual check (**deviation:** `NATIONAL_VALIDATOR` or `NATIONAL_ANALYST`)
- **Purpose:** Corrects a VERIFIED incident's nonsensitive fields (fixed allow-list of ~24 fields, PII intentionally excluded), recomputes `data_hash`, hash-chains the correction.
- **Notable errors:** 403; 404; 409 not VERIFIED; 422 no valid fields; 500 missing hash-chain columns.
- **Touches PII:** no — the field allow-list explicitly excludes PII. **Audit-logged:** yes — hash-chained `incident_verification_history` row (`prev_ivh_hash/ivh_row_hash`) plus `CORRECTION` in `system_audit_trails`.

POST /validator/incidents/bulk-approve, PATCH .../archive, PATCH .../unarchive

- **Auth:** `get_national_validator` throughout; all idempotent via `client_id`, all audit-logged through the shared lifecycle helpers.

DELETE /validator/incidents/{incident_id} — hard delete

- **Auth:** `get_national_validator`
- **Purpose:** Permanently deletes an **archived** incident and all child rows.
- **Touches PII:** yes — the cascade explicitly deletes rows from `wims.incident_sensitive_details`.
- **Audit-logged:** **this is the one place in the reviewed codebase where `incident_verification_history` rows are deleted rather than appended** (it removes the target incident's IVH rows as part of cleanup, logging only `logger.info/logger.exception`, not a new audit row). This stands in tension with IVH's append-only posture used everywhere else and is called out here explicitly rather than smoothed over — it is bounded to archived incidents only, but worth a compliance-team decision on whether that exception should exist at all.

GET /validator/incidents/{incident_id}/diff, /history

- **Auth:** `get_national_validator` — `diff` is explicitly restricted to a fixed non-PII field set; `history` read re-verifies the IVH hash chain on every call. Both read-only.

GET /validator/audit-logs, /audit-logs/export, /audit-logs/export/secure

- **Auth:** `get_national_validator`
- **Purpose:** Validator's own-scoped audit query (RP-25: forcibly scoped to `actor_user_id = self`, cannot query others), CSV export, and a signed tamper-evident ZIP export (OpenBao-signed).
- **Notable errors (secure export):** 413 too large; 503 OpenBao unavailable; 500 if the export's own audit record fails to write — **this is a hard failure, not swallowed**, unlike the plain CSV export whose self-audit failure is logged and ignored while the CSV still downloads.

Fire Incident Perimeters (regional/perimeters.py)

Local gates: `_require_perimeter_editor` (NATIONAL_VALIDATOR/SYSTEM_ADMIN), `_require_perimeter_reader` (adds REGIONAL_ENCODER for reads).

POST /incidents/{incident_id}/perimeter, PUT .../perimeter, DELETE .../perimeter

- **Auth:** `_require_perimeter_editor`
- **Request:** `PerimeterCreateRequest/PerimeterUpdateRequest` { `geometry`: `GeoJSON`, `map_method` }
- **Notable errors:** 400 invalid geometry/method; 409 perimeter already exists (create); 404 no perimeter (update/delete).
- **Audit-logged:** yes throughout (PERIMETER_CREATE/UPDATE/DELETE). A DB trigger closes (not deletes) the perimeter's history row on delete — an append-style pattern mirroring IVH.

GET /incidents/{incident_id}/perimeter

- **Auth:** `_require_perimeter_reader` — read-only, no audit write.

POST /incidents/{incident_id}/link-reports, DELETE .../link-reports

- **Auth:** `_require_perimeter_editor` — links/unlinks civilian reports to an incident. **Audit-logged:** yes (PERIMETER_LINK/PERIMETER_UNLINK).

Triage Queue & Promotion Workflow (triage.py, prefix /api/triage)

Local gates: `_require_encoder_or_validator`, `_require_cluster_workflow_actor`, `_require_status_update_actor` (NATIONAL_VALIDATOR or REGIONAL_ENCODER).

GET /queue

- **Auth:** `_require_encoder_or_validator` — the modern cluster-based triage queue, replacing legacy pending/promote. Read-only.

POST /clusters/{cluster_id}/claim, /activity, GET .../activity, .../merge-candidates, POST .../terminal-action, POST /reports/{report_id}/correct, POST .../split, POST /clusters/{target_id}/merge

- **Auth:** `_require_cluster_workflow_actor` throughout — cluster claim/heartbeat/split/merge/dismiss workflow. All mutating actions are audit-logged via `log_system_audit` inside their respective service commands.

POST /reports/{report_id}/update-status

- **Auth:** `_require_status_update_actor` (**deviation** — the one civilian-facing status-push endpoint encoders can also call)
- **Purpose:** Records a validator/encoder-to-civilian status update, enforcing a forward-only stage lifecycle; publishes an SSE event.
- **Response:** `StatusUpdateResponse` { `update_id`, `report_id`, `stage`, `metadata`, `actor_user_id`, `created_at` } (201). **Audit-logged:** yes.

GET /pending — deprecated, still live. Docstring: “use GET /api/triage/queue instead.” Not yet a 410.

POST /{report_id}/promote

Status: DEPRECATED — returns **410 Gone**. Retired in favor of the cluster-based triage queue.

POST /bulk-promote

Documented discrepancy — verify against the promote route above before treating as equivalent. Despite sitting next to a 410-stubbed sibling, this handler is live and functional: it calls `_bulk_set_status(..., action_type="CIVILIAN_REPORT_PROMOTE")` and sets status to `ACTIONED`. It does not raise 410. Flagged for the team to reconcile intent (retire it to match, or correct the singular endpoint’s deprecation).

POST /bulk-dismiss, POST /bulk-link

- **Auth:** `_require_encoder_or_validator` — bulk status transitions (max 100 reports/batch, all-or-nothing on missing ids). **Audit-logged:** yes, one row per mutated report.

Operational Map (`map.py`, `operational_router`, mounted `/api/validator`)

GET /fire-stations, GET /operational-map

- **Auth:** only `Depends(get_db_with_rols)` is declared in the route signature — no explicit `get_current_wims_user/role` dependency appears in this file. This is flagged for verification rather than asserted as a gap: CLAUDE.md’s documented convention is `get_current_wims_user` before `get_db_with_rols` on every gated route, and it’s possible the actual gating happens via a router-level `dependencies=` argument at the `include_router(...)` call in `main.py` (not visible from this file alone). Confirm before relying on this document’s silence as “this route is unauthenticated” — treat as **unverified, not confirmed-public**.
 - **Purpose:** Fire-station reference list / server-clustered operational incident map (PostGIS `ST_SnapToGrid`) with Redis stale-cache fallback on DB failure.
 - **Touches PII:** no (aggregate clusters only).
-

4.5 Analytics

All routes: `Depends(get_analyst_or_admin)` (`NATIONAL_ANALYST` or `SYSTEM_ADMIN`), except `GET /api/analytics/audit-logs` which uses the strict `get_national_analyst` (excludes admin — RP-25, analysts can only ever see their own actions).

Endpoint	Purpose	Notes
POST /refresh-views	Queue async materialized-view refresh	{task_id, status: "queued"}, 202
GET /heatmap	GeoJSON incident heatmap	filterable by date/geo/type/casualty/damage
GET /trends	Time-series counts	interval: daily/weekly/monthly/quarterly/yearly
GET /comparative	Two-range variance comparison	422 on invalid ranges
GET /execution-plans	EXPLAIN proof of indexed access	operational/diagnostic
POST /export/csv, /pdf, /excel	Dispatch Celery export task	async — returns {task_id} only
POST /export/workflow/comparative, /trends, /response-time, /top-n	Dispatch workflow-specific export	all async, {task_id}
GET /export/{task_id}	Download completed export	FileResponse; 409 pending/failed, 404 missing file — shared download endpoint for every export-dispatch route in this table and in incidents.py
GET /filter-options, /type-distribution, /response-time-by-region, /compare-regions, /top-n	Dashboard aggregate queries	all read-only, no PII
GET /audit-logs	Analyst's own-scoped audit query	get_national_analyst strict; RP-25 self-scoping

Touches PII: the export endpoints (csv/pdf/excel/workflow/*) can include PII-bearing columns depending on ALLOWED_EXPORT_COLUMNS selection — flagged as **yes (conditional)** rather than blanket yes/no. **Audit-logged:** none of the analytics routes write to system_audit_trails or incident_verification_history directly (exports are fire-and-forget task dispatches; the task itself may log independently, not verified in this pass).

4.6 Incidents / Operations / Dashboard / Events

incidents.py

POST /api/incidents/upload-bundle

- **Auth:** get_current_wims_user (region-scoped internally for encoders)
- **Purpose:** Offline/online bulk-import compatibility endpoint; per-item savepoint rollback, client_id idempotency.
- **Request:** IncidentBundleCreate { incidents: [...] (max WIMS_MAX_BUNDLE_SIZE=200), region_id } — each item ~30 nonsensitive fields + the 4-field PII set.
- **Response:** { status, batch_id, imported, incident_ids, failed: [{index, reason}], message }
- **Notable errors:** 422 payload exceeds 5MB; 403 REGION_NOT_ASSIGNED/REGION_MISMATCH; per-item failures captured in failed[] rather than raising (e.g. pii_encryption_unavailable — **fail-closed per item**, not per-batch).
- **Touches PII:** yes. **Audit-logged:** yes (per-incident IVH CREATED_DRAFT + one batch-level UPLOAD_BUNDLE audit row).

POST /api/incidents/{incident_id}/attachments, GET .../attachments/{attachment_id}

- **Auth:** get_current_wims_user (upload); download additionally requires role in {SYSTEM_ADMIN, NATIONAL_ANALYST, NATIONAL_VALIDATOR, REGIONAL_ENCODER}
- **Purpose:** AES-256-GCM-encrypted attachment upload/download (EXIF stripped for images, magic-byte + extension validation, 25MB cap).
- **Notable errors:** 403 role not in viewer set; 404 missing; 500 decrypt failure (“possible tampering or key mismatch”).
- **Audit-logged:** yes, both upload and download (UPLOAD_ATTACHMENT/DOWNLOAD_ATTACHMENT).

POST /api/incidents, GET /api/incidents

- **Auth:** get_current_wims_user (create) / auth.get_incident_viewer — SYSTEM_ADMIN/NATIONAL_ANALYST/NATIONAL_VALIDATOR/REGIONAL_ENCODER (list)
- **Note on GET /api/incidents:** this plain list endpoint returns **plaintext** owner_name, establishment_name, caller_name directly from wims.incident_sensitive_details columns — no decryption performed, no PII-specific audit log on read. For encrypted-only rows these plaintext columns are NULL, but for any legacy row still carrying plaintext PII, this list surfaces it to any of the four gated roles. Documented as-is; flagged as the broadest plain-PII-read surface in the reviewed codebase.

GET /api/incidents/analyst/{incident_id}/sensitive — the dedicated PII-decryption endpoint

- **Auth:** get_analyst_or_admin
- **Purpose:** Decrypts and returns caller_name, caller_number, owner_name, occupant_name, narrative_report, casualty_details, estimated_damage_php for a single VERIFIED incident.
- **Response:** includes pii_decryption_failed: bool.

- **Verified compliance behavior (STEP 0 finding, restated in context):** on any decryption exception, the handler sets `pii_decryption_failed=true` and the PII fields resolve to null — because the underlying plaintext columns for encrypted rows are always NULL, there is no stale-plaintext fallback to leak. When no sensitive-detail row exists at all, all PII fields are null with the flag false. Blob metadata (`pii_blob_enc`, `encryption_iv`, `crypto_provider`, `kms_key_name`, `key_version`) is stripped before response in every case.
- **Audit-logged: no** — this read-only PII-decrypting endpoint does not itself write to `system_audit_trails` or `incident_verification_history`. Flagged as a gap worth compliance-team attention: every other PII-touching read/write in this document either has an audit trail or is scoped to a role that's separately audited elsewhere; this one is not.

GET /api/incidents/analyst-list, /analyst/{id}, /analyst/{id}/wildland, POST /analyst/export/{format}

- **Auth:** `get_analyst_or_admin` throughout — national analyst dashboards, all explicitly excluding `incident_sensitive_details` joins (the `/sensitive` endpoint above is the sole exception by design).

operations.py

All mutating routes: `get_national_validator`. GET `/api/operations` uses `get_incident_viewer` (four internal roles) — code comment notes this was previously unauthenticated (flagged internally as EP-29/audit gap #3, now closed). CRUD + link/unlink + daily reset/restore workflow for the fire-status operations board; every mutating route writes a distinct audit action (`OPERATION_CREATE/UPDATE/DELETE/LINK_REPORT/UNLINK_REPORT/DAY_RESET/RESTORE`). No PII fields touched.

dashboard.py

GET /api/dashboard/widgets

- **Auth:** `get_current_wims_user`, with **per-widget** role gating from a static `WIDGET_DEFS` map (requested widget ids outside the caller's role set are silently skipped, not rejected).
- **Notable finding:** per-widget SQL failures return `{"error": str(exc)}` inline for that widget rather than failing the whole request — raw exception text in a response body is a minor info-disclosure consideration worth a look, noted rather than fixed (doc-only round).

events.py

GET /api/events/stream (SSE)

- **Auth:** performed **manually inside the handler**, not via `Depends(...)` — calls `await get_current_user(request)` directly, then layers a second, independent channel-authorization check (`_ROLE_CHANNEL_MAP`: encoders/validators get `incident+verification`; analysts get `incident` only; admins get all four channels; civilian reporters get none). This dual-layer, manual-auth pattern is unique to this endpoint in the reviewed codebase, driven by the need to reject cleanly before entering a long-lived streaming response.
- **Notable errors:** 401 auth failure/no role; 400/403 unknown or disallowed channel.
- **Audit-logged:** no (only application logs).

4.7 Admin

Nineteen sub-routers mounted under `/api/admin`. **Universal auth pattern: every endpoint in this domain is gated by `Depends(get_system_admin)`**, with exactly one documented exception — `admin/sync.py`'s `POST /sync/report`, which intentionally accepts any authenticated role (`get_current_wims_user`) because it's a client-side sync-failure beacon, not an admin action.

User & Session Management (`users.py`, `sessions.py` — sessions covered in §4.1)

`POST /users` (onboard — creates Keycloak user, never returns the temp password; 409 if exists), `POST /users/{keycloak_id}/resend-credentials`, `GET /users` (list, Keycloak IDs masked), `PATCH /users/{user_id}` (role/region/active-flag only — no direct field edit for name/email; deactivation revokes all sessions), `GET /active-sessions`, `POST /users/{user_id}/logout` (force logout — audited as `LOGOUT` with `initiated_by: admin_force_logout`, RP-19 non-repudiation).

IP / Device Blocklists (`ip_blocklist.py`, `device_blocklist.py`)

`GET/DELETE /ip-blocklist[/ip]`, `GET/DELETE /device-blocklist[/token_hash]` — list and soft-unblock. Device identifiers are always hashed tokens, never raw.

Community CMS (`community.py` → `community_content.py::admin_router`)

`GET/POST /community`, `PATCH /community/{id}`, `POST /community/{id}/publish`, `POST /community/{id}/archive` — draft/publish/archive lifecycle with optimistic-concurrency (409 on version conflict) and no physical delete (archive only).

Analytics Admin (`admin/analytics.py`)

`POST /analytics/backfill` — rebuild `analytics_incident_facts` from verified incidents. **Not** audit-logged.

Rate Limit Config (`rate_limits.py`)

`GET/PATCH /rate-limits` — read/update the login-tier Redis rate-limit config (only tier currently supported: "login"). `PATCH` is audited (`RATE_LIMIT_UPDATED`).

Breach Notifications (`breach.py` — RA 10173, M10d)

`GET /breach`, `GET /breach/{id}`, `PATCH /breach/{id}` — the regulatory breach-tracking workflow; `PATCH` auto-stamps `npc_submitted_at` on transition to `NPC_SUBMITTED` and is audit-logged with old/new snapshots. Fed automatically by `admin/security.py`'s HITL confirm-threat flow (see below) — a HIGH/CRITICAL confirmed threat creates a breach row here with a 72-hour NPC deadline.

Scheduled Reports (`scheduled_reports.py`)

`GET/POST /scheduled-reports`, `PATCH/DELETE /scheduled-reports/{id}` — cron-driven analytics export configuration. Not audit-logged (config CRUD on a non-PII-bearing table).

System Config (`config.py` — M9c)

`GET /config`, `PATCH /config/{key}` — key allow-listed against `VALID_CONFIG_KEYS` (rejects arbitrary key injection), with per-key numeric range checks and cross-key consistency rules; fires

a `system.config_changed` Redis pub/sub event on commit. Audited (`CONFIG_UPDATE`, old/new snapshot).

Anomaly Detection (`anomalies.py`)

GET `/anomalies`, PATCH `/anomalies/{id}` — behavioral anomaly lifecycle, strict state machine `NEW` → `ACKNOWLEDGED` → `RESOLVED` (409 on an invalid transition). **Touches PII:** yes (`subject_user_id` links each anomaly to the user whose behavior triggered it). Status changes are audited (`ANOMALY_ACK/ANOMALY_RESOLVE`).

Audit Trail (`audit.py`)

GET `/audit-logs` (filterable, full-text search over `system_audit_trails`; **not** itself audited on read), GET `/audit-logs/export` (CSV; **self-referentially audited** — `AUDIT_EXPORT`, per RP-23: “a `SYSTEM_ADMIN` cannot deny exporting sensitive audit data”), GET `/audit-logs/export/secure` (signed tamper-evident ZIP via OpenBao; **hard-fails 500** if its own audit record can’t be written — unlike the plain CSV export, which logs-and-continues on the same failure), POST `/audit-logs/export/verify` (verify a previously-downloaded signed package; 100MB cap). POST `/audit-logs/analyze` (LLM-driven) is **excluded**, see §8.

Offline Sync Monitoring (`sync.py`)

GET `/sync/failed` (in-memory, non-persistent — explicitly noted in the file itself: “Production would store this in PostgreSQL”), POST `/sync/report` (**the one non-admin-gated route in this domain**, `get_current_wims_user`), POST `/sync/{op_id}/retry`, DELETE `/sync/{op_id}`.

Civilian Contributor Management (`civilians.py` — issue #576)

GET `/civilians` (search/status filter — email is always `None`, sourced from Keycloak but not joined here), POST `/civilians/{user_id}/suspend/activate` (idempotent — no re-audit on a no-op call; best-effort Keycloak sync), GET `/civilians/{user_id}/audit` (per-individual audit view — itself PII-adjacent).

Privacy — RA 10173 Data Subject Rights (`privacy.py`) — compliance-critical, documented in full

GET `/privacy/export`

- **Purpose:** The canonical PII-export endpoint. `subject_type=USER` returns the user’s own profile + consent history (no incident PII — third-party data is excluded from a user-subject export by design). `subject_type=REPORT` returns the `citizen_reports` row (witness fields decrypted from `witness_pii_blob_enc`) plus linked `incident_sensitive_details` (decrypted `caller_name/caller_number/owner_name/occupant_name/narrative_report/casualty_details`) plus consent history.
- Response sets Cache-Control: no-store, no-cache, must-revalidate + Pragma: no-cache. Decryption failures are logged and surfaced as `decryption_failed: true` in the payload — consistent with the fail-closed pattern verified in §2/STEP 0, not silently omitted. Blob metadata columns are always stripped.
- **Audit-logged:** yes (`PII_EXPORT`) — the *access* is audited; the exported payload’s contents are not themselves logged.

POST /privacy/anonymize

- **Purpose:** The right-to-erasure counterpart. Irreversibly nulls PII in place (no row deletion, preserving FK integrity). USER subject: nulls contact_number only (username is preserved — it's referenced from audit trails). REPORT subject: refuses with 409 unless the report is in a terminal status; nulls witness fields + encrypted blob on citizen_reports, all PII columns + blob on incident_sensitive_details, and full_name on every involved_parties row for the linked incident.
- Idempotent — repeat calls on already-nulled rows succeed silently (no duplicate audit rows).
- **Response:** { anonymized: true, subject_type, subject_id, tables_affected, warning: "irreversible" } — the response body itself states there is no recovery path.
- **Audit-logged:** yes — one PII_ANONYMIZE row per affected table, only where a row actually changed.

Security Telemetry / HITL Review (security.py)

GET /security-logs (filterable list, excludes dismissed by default), /summary, /rollups (hourly/daily pre-aggregated), POST /security-logs/block-by-filter (bulk IP block; audited **only when not a dry-run** — preview=true performs no writes and no audit), POST /security-logs/bulk-action (block-ip/block-device/dismiss/false-positive across selected ids — one audit row per batch, per-item failures tolerated inline), PATCH /security-logs/{id} (HITL decision: CONFIRM_THREAT/FALSE_POSITIVE/REQUEST_MORE_INFO — a HIGH/CRITICAL confirmed threat **automatically creates a breach_notifications row** with a 72-hour NPC deadline and emails all active SYSTEM_ADMIN users, bypassing individual email_opt_in for this one regulatory-notification case), POST /security-logs/{id}/block-source-ip, POST /security-logs/{id}/block (device-or-IP, type param), POST /security-logs/bulk-block-preview (read-only dry-run grouping, never mutates), POST /security-logs/{id}/create-incident (promotes a reviewed alert to a DRAFT fire incident; 409 if one already exists), DELETE /security-logs/{id} (soft-delete/dismiss only), GET /security-logs/{id}/related-audit (cross-references the audit trail ±1hr window — returns rows that may carry PII from other domains via JSONB snapshots).

Four LLM/Ollama-driven endpoints in this file (analyze-status, analyze, recommended-action-status, recommended-action) are **excluded**, see §8.

Information CMS (admin/information.py, prefix /information)

GET/POST /information/announcements, PUT/DELETE /information/announcements/{id}, GET/POST /information/emergencies, PUT/DELETE /information/emergencies/{id}, POST /information/emergencies/promote/{incident_id} (creates/refreshes a public emergency draft from a verified incident). Module docstring notes these CMS tables have no RLS policies — access control is get_system_admin alone. Not audit-logged in this file (content CRUD, non-PII).

System Health Dashboard (admin/monitoring.py)

GET /health — admin-facing composite health check (database/redis/keycloak/suricata/ollama reachability); documented in full since it's a SYSTEM_ADMIN UI dashboard, not a raw metrics dump. Three sibling endpoints in this file (GET /monitoring/workers, POST /monitoring/workers/prune, GET /monitoring/system) are **excluded**, see §8.

5. Data Handling & Compliance Appendix

This section complements — and deliberately does not duplicate — the existing DPA-focused documents at docs/compliance/DPIA.md, docs/compliance/RoPA.md, and docs/compliance/data-retention.md. Those cover the RA 10173 legal/process framing; this section maps that framing onto the concrete API surface documented above.

5.1 PII map — which endpoints touch personal data

The four named PII fields are `caller_name`, `caller_number`, `owner_name`, `occupant_name`, encrypted as a single AES-256-GCM JSON blob in `wims.incident_sensitive_details.pii_blob_enc` (plus `narrative_report`, `casualty_details`, and `estimated_damage_php`, which travel in the same blob though not part of the original four-field set). Witness PII (`witness_name`, `witness_phone`) on `wims.citizen_reports` follows the same encryption pattern via a separate `witness_pii_blob_enc` column.

	Endpoint	PII touched	Direction
POST	/api/v1/public/report	none (description field only, no structured PII fields)	—
POST	/api/civilian/reports,/append	witness_name, witness_phone	write (encrypt; not fail-closed — falls through to plaintext on KMS failure)
POST	/api/regional/incidents, PUT/PATCH .../incidents/{id}	all four PII fields + narrative/casualty	write (encrypt; fail-closed — PiiEncryptionUnavailable blocks the write if PII is present and crypto is unavailable)
POST	/api/regional/afor/commit	all four PII fields	write (batch, encrypted)
GET	/api/regional/incidents, GET /api/regional/incidents/{id}	all four PII fields	read (decrypt)
	GET /api/incidents	owner_name, establishment_name, caller_name	read (plaintext columns, no decryption — see §4.6 finding)
GET	/api/incidents/analyst/{id}/sensitive	all four PII fields + narrative/casualty	read (decrypt; the sole dedicated analyst PII-read endpoint)
POST	/api/incidents/upload-bundle	all four PII fields	write (encrypt; fail-closed per item)
POST	/api/analytics/export/*, POST /api/incidents/analyst/export/{format}	conditional, per selected export columns	read (export)
	admin/civilians.py routes	usernames/keycloak IDs (identity, not the four incident PII fields)	read
GET	/api/admin/privacy/export	all four PII fields + witness PII	read (decrypt, the canonical RA 10173 export)
POST	/api/admin/privacy/anonymize	all four PII fields + witness PII + involved_parties.full_name	erase (irreversible)
DELETE	/validator/incidents/{id} (hard delete)	all four PII fields	destroy (cascade delete, not encrypt/decrypt)

5.2 Verified security behavior — PII decryption fails closed

This claim was verified against the actual decrypt/error-handling code, not against comments, for this document:

- **Write path** (`incidents.py::upload_incident_bundle`, and the regional-encoder create/update paths): if PII fields are present in the request but the crypto provider is unavailable (`SecurityProviderError`), the write is refused — `PiiEncryptionUnavailable` — rather than falling back to a plaintext write. Confirmed by `src/backend/tests/test_pii_encryption_fail_closed.py::TestT`
- **Read path** (`incidents.py::get_analyst_incident_sensitive_detail`): on any decryption exception, the handler sets `pii_decryption_failed = True` and the response's PII fields resolve to `None`. There is **no stale-plaintext fallback** — the plaintext columns underlying an encrypted row are always `NULL` by design, so a decryption failure simply surfaces nulls with the failure flag set, never legacy data. Confirmed by `test_pii_encryption_fail_closed.py::TestAnalystSensitivePII::te`
Legacy pre-encryption rows (plaintext columns still populated, no blob) are returned as-is — this is not a “failure” case, it's the documented dual-read path for rows written before encryption was introduced.
- The one documented exception to this clean fail-closed picture is `POST /api/civilian/reports` and `/append`: witness PII encryption failure there is **not** fail-closed — it logs a warning and writes plaintext witness fields instead. This is a deliberate availability trade-off (civilian intake must not be blockable by a KMS outage) but is a materially different posture from the encoder/analyst path above, and should be stated as such rather than folded into a single blanket “PII always fails closed” claim.

5.3 Data classification — encrypted vs. plaintext boundary

Field class	Storage	Notes
caller_name, caller_number, owner_name, occupant_name, narrative_report, casualty_details, estimated_damage_php	AES-256- GCM blob (pii_blob_enc + encryption_iv)	Plaintext columns of the same name always NULL for new writes; populated only on pre-encryption legacy rows
witness_name, witness_phone (civilian re- ports)	AES-256-GCM blob (witness_pii_blob_enc)	Same pattern, separate blob/ table
street_address, landmark, establishment_name, receiver_name, disposition/ officer fields	plaintext columns on incident_nonsensitive incident_sensitive_details	Not encrypted — operational/ location data, not treated as the protected PII set
owner partially (regional) (coder field-update path only)	field_updates.py::_apply_incident_updates mirrors owner_name to its own plaintext column “used by list queries,” in addition to the encrypted blob	It is the one PII field in the four-field set that is not blob- only end-to-end; flagged here since it’s a narrower guaran- tee than the other three
Attachments (incident_attachments)	AES-256-GCM per-file en- cryption on disk	EXIF stripped from images before storage

5.4 Audit logging coverage

Two append-only(-by-design) audit surfaces exist: `wims.system_audit_trails` (general action log) and `wims.incident_verification_history` (incident-lifecycle log, hash-chained via `prev_ivh_hash/ivh_row_hash` for tamper evidence, integrity re-verified on every read).

Domains that write to one or both, per the per-domain sections above: Auth & Session (registration, email/password change, force-logout, security events), Civilian/Public Intake (submission, quarantine flags, `PUBLIC_INCIDENT_SUBMIT` — sharing a transaction with the incident insert, so audit failure rolls back the submission itself), Regional Encoder (create/edit/submit/archive — **except** the archive endpoint, a documented gap), Validator/Triage/Perimeters (verification, correction with hash-chaining, perimeter CRUD, cluster workflow, bulk actions), Operations (every mutating route), and most of Admin (user management, config, breach, anomalies, security HITL, civilian suspension, the privacy export/anonymize pair).

Two gaps surfaced by this pass, stated plainly rather than smoothed over: 1. `GET /api/incidents/analyst/{id}/sensitive` — the single endpoint that decrypts and returns all four PII fields to an analyst — does **not** write an audit row on access. Every other PII-touching endpoint in this document either audits the access directly or is covered by a broader audit on the same

transaction; this one is not. 2. `PATCH /api/regional/incidents/{id}/archive` performs a raw `UPDATE` with no corresponding `system_audit_trails` or `incident_verification_history` write, unlike its sibling `unarchive` endpoint which is audited via the shared lifecycle service.

One append-only exception was also found and is worth a compliance-team decision rather than a silent fix: `DELETE /validator/incidents/{incident_id}` (regional validator hard-delete of an archived incident) removes that incident's `incident_verification_history` rows as part of its cleanup cascade, rather than appending a final “deleted” entry. This is the sole place in the reviewed route files where IVH rows are removed rather than appended.

5.5 Auth mechanism summary (cross-reference to §2)

Keycloak `OIDC/PKCE`; `HttpOnly` `__Host--`-prefixed cookie (literal code read: `request.cookies.get("access_token")`), never an `Authorization` header; JWKS fetched from the Docker-internal Keycloak host, `iss` claim validated against a separately-configured browser-visible issuer (§2.2, verified against code — a deliberate Docker-networking accommodation, not a vulnerability); Redis-backed instant session revocation with a 12-hour TTL, fail-open on Redis unavailability.

6. Follow-Up Work (deferred, out of scope for this PR)

This PR is documentation-only — **no route code, schemas, or response_model annotations were changed**. The prior scope-out identified that roughly half of all route files (validator, analytics, most of admin) carry zero `response_model` annotations, which is why this document hand-documents response shapes from source rather than relying on `/openapi.json`. Recommended follow-up, tracked separately:

1. Add `response_model=` to `regional/validator.py`, `analytics.py`, and the `admin/*` routers so `/openapi.json` becomes a complete machine-readable contract instead of a structural scaffold.
2. Resolve the `bulk-promote` vs. `promote` deprecation discrepancy in `triage.py` (§3, §4.4) — one team decision, not a doc fix.
3. Decide whether `PATCH /api/regional/incidents/{id}/archive` and `GET /api/incidents/analyst/{id}/sensitive` should gain audit-trail writes to close the two gaps noted in §5.4.
4. Decide whether the `incident_verification_history` hard-delete in `DELETE /validator/incidents/{incident_id}` (§4.4, §5.4) should be replaced with an append-only “deleted” terminal entry, consistent with the rest of the system’s IVH posture.
5. Confirm the actual auth gating on `map.py`’s `operational_router` routes (§4.4) — this document could not confirm from the route file alone whether a role dependency is applied at router-mount time in `main.py`.

None of the above required code changes to produce this reference; they are flagged here as findings from reading the code, for the team to act on separately.

7. Existing Spec Status

`/docs`, `/redoc`, and `/openapi.json` are live at FastAPI's defaults (no `docs_url`/`openapi_url` overrides in `main.py`). `/health` and `/metrics` are excluded from the schema (`include_in_schema=False`) and from this document (§8). A live export of `/openapi.json` was not obtained for this document — the pinned dependency `fastapi>=0.135.0` in `src/backend/requirements.txt` does not resolve against the public PyPI index available in the authoring environment (max available: 0.128.8), and no Docker daemon was available to build the full stack instead. This document was built entirely from static source analysis (route decorators, Pydantic schemas, and dependency functions read directly), which — given the uneven `response_model` coverage described above — would have been necessary regardless of whether a live export succeeded, since a live export alone would not have captured response shapes for the un-annotated half of the surface.

8. Endpoints Intentionally Excluded

The following exist in the codebase but are deliberately **not** documented in full above, to avoid publishing internal operational/attack surface with no benefit to external consumers or compliance reviewers. Each is confirmed to exist and gated as noted; request/response contracts are withheld.

Endpoint(s)	Gating	Reason for exclusion
GET /health, GET /metrics	unauthenticated	include_in_schema=False already; liveness/scrape endpoints, no data surface
POST /api/auth/keycloak-event	Bearer shared-secret (server-to-server)	Keycloak SPI event ingest, not a client-facing contract
admin/backups.py (6 routes: trigger/list/download/delete/manifest/restore)	get_system_admin	Backup/restore tooling — publishing this documents a destructive-operation surface with no external-consumer benefit
admin/backup_schedule.py (2 routes)	get_system_admin	Same rationale — backup scheduling config
admin/security.py: analyze, analyze-status, recommended-action, recommended-action	get_system_admin	Ollama/LLM-driven internal analysis tooling, not a stable API contract
admin/audit.py: POST /audit-logs/analyze	get_system_admin	Same — LLM-driven analysis
admin/monitoring.py: GET /monitoring/workers, POST /monitoring/workers/prune, GET /monitoring/system	get_system_admin	Raw infrastructure/worker metrics — GET /health (a composite dashboard) is documented normally in §4.7; these three raw-metrics/ops-maintenance routes are not
POST /api/triage/{report_id}/promote	get_current_wims_user variant	Returns 410 Gone unconditionally — retired, listed in §3/§4.4 as deprecated rather than given full treatment
GET /api/civilian/contributor/leaderboard	PUBLIC	Returns 410 Gone unconditionally — removed feature
GET /api/civilian/reports (bare list)	PUBLIC	Returns 410 Gone unconditionally — retired legacy tracking

No dead/unmounted route files (e.g. a legacy top-level regional.py duplicating regional/validator.py/encoder.py, flagged as a risk in the pre-work scope-out) were found in this codebase snapshot — verified absent via direct file-glob, not assumed.